

Análise Sênior do Sistema de Chats com IA

Revisão técnica da feature de RAG por reunião no Notura

Data: 01/05/2026

Escopo: rotas de chat, job Inngest, RAG, Gemini, migrations Supabase, persistência

de chats e chunks.

Arquivos principais: `src/app/api/meetings/[id]/chats/*`, `src/inngest/answer-`

`meeting-chat.ts`, `src/lib/meetings/rag.ts`, `src/lib/gemini.ts`,

`supabase/migrations/013_meeting_rag_chat.sql`.

Resumo Executivo

A implementação é um bom primeiro corte: respeita a arquitetura backend-first do projeto, separa a rota síncrona do processamento de IA via Inngest, persiste perguntas/respostas para futuro cache semântico e criou uma base de RAG compreensível. Para um júnior, a direção está correta.

Como revisão sênior, porém, eu não liberaria isso para produção com tráfego real ainda. O sistema tem riscos relevantes em segurança Supabase/RPC, abuso de custo, idempotência, concorrência de indexação, vazamento de erro técnico e robustez contra prompt injection. O mais grave é a função SQL `security definer` receber `p_user_id` do cliente e estar liberada para `authenticated`; isso pode permitir leitura indevida de chunks se alguém souber ou descobrir IDs.

Veredito: a feature é promissora, mas ainda está em estágio de MVP técnico. Antes de habilitar amplamente, corrija primeiro RPC/RLS, rate limit/quota, idempotência do envio Inngest e atomicidade da indexação.

Arquitetura Atual

1. O frontend chama `POST /api/meetings/[id]/chats` com uma pergunta curta.
2. A rota autentica, valida ownership da reunião, valida pergunta, cria uma linha em `meeting_chats` com status `processing` e dispara o evento `meeting/chat.answer`.
3. O job `answerMeetingChat` carrega chat e reunião via service role, garante chunks indexados, gera embedding da pergunta, chama a RPC de match vetorial e pede ao Gemini uma resposta baseada nos chunks.
4. O resultado final é salvo na linha do chat: resposta confirmada, fallback de baixa similaridade, fallback por não confirmação do modelo ou falha técnica.
5. O frontend consulta `GET /api/meetings/[id]/chats/[chatId]` até status final.

O que está bom: a rota não chama IA de forma síncrona, o desenho de "um chat = uma pergunta" mantém contexto controlado, há metadados de chunks, HNSW foi escolhido corretamente para busca vetorial, e os testes cobrem os principais fluxos felizes e fallbacks.

Achados Principais

Severidade	Achado	Evidência	Impacto
Crítica	RPC <code>security_definer</code> confia em <code>p_user_id</code> informado pelo chamador.	<code>013_meeting_rag_chat.sql:70-117</code>	Um usuário autenticado pode tentar chamar a RPC diretamente com IDs de outro usuário e receber texto de chunks, bypassando RLS.
Alta	Polícies <code>for all</code> permitem mutação direta das tabelas de chat/chunks por usuários autenticados.	<code>013_meeting_rag_chat.sql:60-68</code>	Possível envenenamento de cache, alteração/deleção de chunks próprios e inconsistência entre backend e dados.
Alta	Não há rate limit, quota ou controle de custo por usuário/reunião no endpoint de criação.	<code>route.ts:23-76</code>	Usuário autenticado pode gerar muitas chamadas Gemini/Inngest, aumentando custo e risco de rate limit externo.
Alta	Falha ao disparar Inngest deixa chat em <code>processing</code> .	<code>route.ts:51-76</code>	Após inserir o chat, se <code>inngest.send</code> falhar, a linha pode ficar presa sem resposta final.

Severidade	Achado	Evidência	Impacto
Alta	Indexação apaga chunks antes de garantir novos embeddings.	<code>rag.ts:150-165</code> , <code>rag.ts:326-355</code>	Uma falha parcial de provider pode remover chunks existentes e quebrar buscas futuras daquela reunião.
Média	Concorrência de backfill pode causar inserts duplicados ou condição de corrida.	<code>answer-meeting-chat.ts:185-187</code> , <code>rag.ts:170-190</code>	Dois chats simultâneos para reunião antiga podem tentar indexar ao mesmo tempo e conflitar com <code>unique (meeting_id, chunk_index)</code> .
Média	Backfill perde metadados ricos de AssemblyAI quando só existe transcript formatada.	<code>process-meeting.ts:317-325</code> , <code>rag.ts:182-188</code> , <code>rag.ts:300-313</code>	Reuniões antigas podem virar chunks pobres ou um único chunk grande, reduzindo qualidade da resposta e fontes.
Média	Embeddings de chunks são gerados em paralelo sem limite.	<code>rag.ts:338-356</code>	Reuniões longas podem disparar muitas chamadas simultâneas ao Gemini, gerando 429, latência e custo imprevisível.
Média	Job Inngest está com	<code>answer-meeting-</code>	Erros transitórios

Severidade	Achado	Evidência	Impacto
	<pre>retries: 0.</pre>	<pre>chat.ts:158-162, gemini.ts:316-331</pre>	viram falha final rapidamente; a feature fica frágil contra instabilidade de provider.
Média	Prompt de resposta exige JSON, mas o exemplo fixa <pre>is_answered_from_context: true.</pre>	<pre>gemini.ts:215-230</pre>	Isso incentiva o modelo a confirmar mesmo quando deveria retornar falso, enfraquecendo o fallback de segurança.
Média	Erros técnicos são persistidos e expostos no GET.	<pre>answer-meeting- chat.ts:144-154, [chatId]/route.ts:5-17</pre>	Pode vazar mensagens internas de provider, Supabase, modelo ou configuração para o cliente.
Baixa	Parsing de JSON do Gemini usa regex gulosa como fallback.	<pre>gemini.ts:337-344</pre>	Resposta com chaves extras ou texto inesperado pode gerar parse incorreto; melhor usar JSON mode/schema quando disponível.

Segurança e Privacidade

1. Corrigir RPC antes de qualquer rollout

A RPC `match_meeting_transcript_chunks` está como `security definer` e recebe `p_user_id`. Esse padrão é perigoso porque o chamador não deve poder dizer ao banco "eu sou tal usuário". Como a função retorna `text` dos chunks, o impacto é leitura de transcrição, que é dado sensível.

Recomendação: remover `p_user_id` da assinatura e usar `auth.uid()` dentro da função; ou transformar em `security invoker` e deixar RLS filtrar; ou revogar `execute` de `authenticated` e chamar apenas via service role. Para este projeto, eu escolheria service role + revogação, porque a arquitetura já proíbe Supabase no frontend.

2. RLS deve ser least privilege

As policies `for all` são amplas demais para tabelas que guardam transcript chunks, embeddings, fontes e respostas. Mesmo quando o usuário só altera dados próprios, ele pode corromper a base que depois alimentará cache semântico, analytics e suporte.

Recomendação: trocar por policies separadas e restritivas. Idealmente, `authenticated` não deveria ter `insert/update/delete` nessas tabelas. O backend com service role escreve; usuários leem via Route Handler.

3. Proteger contra abuso de custo

A rota valida tamanho da pergunta, mas não limita frequência. Em IA, isso é insuficiente: uma pergunta barata na API ainda aciona embedding, busca e geração. Sem quota, um usuário autenticado pode transformar um endpoint legítimo em vetor de custo.

Recomendação: limite por minuto, limite diário por plano, orçamento por conta, bloqueio por reunião em processamento e telemetria de custo por chat.

Confiabilidade e Operação

1. Evitar chats presos em `processing`

O endpoint cria a linha antes de enviar o evento. Esse fluxo é aceitável, mas precisa tratar falha no envio. Hoje, se `inngest.send` falhar, o chat pode ficar sem fechamento operacional.

Recomendação: envolver o envio em `try/catch` e marcar o chat como `failed`; ou usar padrão outbox: cria chat e evento pendente numa transação, e um worker confiável entrega o evento.

2. Indexação deve ser idempotente e atômica

A função apaga chunks e depois cria novos embeddings. Esse é o tipo de fluxo que funciona em teste unitário e falha feio em produção quando o provider oscila. Além disso, dois jobs concorrentes podem observar "sem chunks" e indexar ao mesmo tempo.

Recomendação: gerar embeddings antes de apagar dados antigos; usar tabela temporária ou versão de índice; fazer `upsert` por `(meeting_id, chunk_index)`; aplicar advisory lock por `meeting_id` durante backfill; e registrar `embedding_model`, `embedding_dimensions` e versão do chunking.

3. Retries precisam distinguir erro transitório de erro definitivo

O job está com `retries: 0` e o wrapper Gemini tenta só duas vezes localmente. Isso evita duplicidade, mas sacrifica confiabilidade. O melhor desenho é tornar as escritas idempotentes e permitir retry do job.

Recomendação: habilitar retries do Inngest para erros transitórios; salvar falha definitiva apenas depois das tentativas; manter idempotência no update do chat e na indexação.

Qualidade do RAG

1. Chunking precisa de tokenizer real e overlap

O chunking usa contagem por espaços como proxy de tokens e não tem overlap. Isso é simples e suficiente para MVP, mas perde contexto nas bordas e pode variar muito por idioma, pontuação e transcrição ruidosa.

Recomendação: usar tokenizer compatível com o modelo ou heurística calibrada por caracteres/tokens; adicionar overlap pequeno entre chunks; preservar utterance IDs; e considerar incluir chunk anterior/posterior quando a resposta depender de contexto.

2. Avaliar empiricamente o limiar 0.6

O limiar de 0.6 e `p_limit = 5` foram implementados, mas não há suite de avaliação. Sem dataset de perguntas esperadas, o time não sabe se o problema é baixa similaridade, chunking ruim, prompt ruim ou embedding incompatível.

Recomendação: criar um pequeno conjunto de reuniões sintéticas e reais anonimizadas com perguntas/respostas esperadas. Medir recall@5, taxa de `low_similarity`, taxa de `not_confirmed_by_model` e alucinação.

3. Prompt deve separar dados de instruções

A transcrição é conteúdo não confiável. Um participante pode literalmente dizer "ignore as instruções anteriores". O prompt atual tem boas regras, mas injeta chunks como texto puro. Isso deixa a barreira contra prompt injection mais fraca.

Recomendação: delimitar os chunks como dados, declarar que instruções dentro dos chunks devem ser tratadas como fala da reunião, exigir citação de `chunkId`, corrigir o schema para permitir explicitamente `true` ou `false`, e usar response schema/JSON mode quando o SDK suportar.

Dados, Cache Futuro e Manutenibilidade

A decisão de persistir `question_embedding`, resposta, fontes e fallback prepara bem o caminho para cache semântico. Mas isso também aumenta responsabilidade com retenção, auditoria e consistência.

- Guardar `embedding_model`, dimensão e versão do prompt por chat e por chunk.
- Não armazenar fonte textual duplicada em `sources` se o `chunkId` já resolve; ou armazenar snapshot com política clara de retenção.
- Adicionar status mais operacional: `queued`, `processing`, `completed`, `failed`, `expired`.
- Registrar métricas por etapa: tempo de embedding, quantidade de chunks, similaridade

máxima, tokens aproximados, custo estimado e motivo de fallback.

- Separar erro público de erro interno: cliente recebe mensagem genérica e suporte usa `traceId`.

Plano de Correção Recomendado

Antes de produção

1. Corrigir RPC/RLS: remover confiança em `p_user_id`, revogar execução direta se possível e restringir policies.
2. Adicionar rate limit e quota de IA por usuário/plano/reunião.
3. Tratar falha de `inngest.send` para não deixar chat preso.
4. Tornar indexação idempotente, com lock por reunião e sem apagar chunks antes de novos embeddings existirem.
5. Sanitizar `errorMessage` exposto no GET.
6. Corrigir o prompt de resposta para não fixar `is_answered_from_context: true` no formato obrigatório.

Próxima iteração

1. Persistir utterances normalizadas de AssemblyAI para backfill fiel.
2. Adicionar limite de concorrência/batching para embeddings.
3. Criar suite de avaliação RAG com perguntas esperadas.
4. Adicionar observabilidade por etapa e dashboards de fallback/custo.
5. Versionar modelo, embedding, chunking e prompt nos dados.

Conclusão

O júnior acertou o esqueleto: backend assíncrono, persistência de conversa, RAG simples, fallbacks e testes. O que falta é maturidade de produção: isolamento de banco, controle de abuso, idempotência e avaliação de qualidade.

Minha orientação seria não jogar fora a implementação. Eu manteria o desenho, mas faria uma rodada de hardening antes de integrar UI final e abrir para usuários. O sistema está perto de virar uma feature forte; só não pode atravessar a linha de produção carregando uma RPC permissiva e uma indexação que pode se autossabotar quando o provider piscar.